

Control-M SaaS Scheduling

Designing workflows in the Planning domain: folders, jobs, scheduling criteria, calendars, prerequisites, actions, and jobs-as-code.

1. Building blocks

A **workflow** is a set of connected jobs that run at set times, in a set order, once their prerequisites are met. Jobs are the basic execution unit: a script or command on a host, or a call into an application through a plug-in (SAP, file transfer, a database, a cloud service).

Object	Purpose
SMART folder	Container whose scheduling, prerequisites and actions apply to the folder as a whole; has its own run ID and status
Sub-folder	Lives inside a SMART folder and inherits its scheduling definitions
Regular folder	Simple container; each job is processed independently by its own definition
Job	Unit of work. Tabs: General, Scheduling, Prerequisites, Actions
Calendar	Reusable schedule (Regular, Periodic, Rule-Based) shared by many jobs
Event	Condition that links a predecessor to a successor job
Lock resource / Resource pool	Control concurrency: exclusive or shared access, or a quantity limit
Variable	Value passed into or between jobs

Recommendation

Use SMART folders for almost everything. They give you one status for the whole flow, folder-level notifications, and a clean unit to order, rerun and monitor.

2. Workspaces: the change process

All editing in the Planning domain happens inside a **workspace**, a private working copy.

1. Create a new workspace or **check out** existing folders into one.
2. Edit jobs and folders. Validation and Site Standards flag problems as you work.
3. **Check in** to publish the definitions to the Control-M/Server. Nothing changes in production until you check in.
4. Optionally **Run** the workspace or selected folders to order them now instead of waiting for New Day.

Checking in changes the definition only. Jobs already ordered today keep running with the definition they were ordered with, unless you also edit them in Monitoring.

3. Scheduling criteria

The Scheduling tab decides whether a job is ordered on a given day, and the time window in which it may start.

Which days

Option	Behaviour
Every Day	Ordered every day
Use Parent Schedule	Follows the Rule-Based calendar(s) of the parent SMART folder
Specific Dates	Only on the dates you pick
Advanced: Run on Week Days	e.g. Mon-Fri; or day-in-week, such as D3W2 = 3rd day of the 2nd week
Advanced: Run on Month Days	e.g. 1st and 15th; L1 = last day of month; -L1 = all days except the last
Advanced: Months	Restrict to chosen months
Use / Exclude calendars	Include one or more Rule-Based calendars, then exclude others (e.g. holidays)
None	Never ordered automatically; run manually only

When you combine a calendar with week or month days, you can choose how non-working days are handled: run anyway, run only on working days, or move to the next (>) or previous (<) working day.

Which time

Attribute	Meaning
From	Earliest start time. No Start Time = start as soon as prerequisites are met
To	Latest start time. End of Day (New Day time), a set time, or Allow Submission Past Next New Day (>)
Time Zone	Run the job in another zone's clock, e.g. 09:00 Pacific/Honolulu regardless of Agent location
Keep Active	Extra days (0-98, or Forever) the job waits if it could not run on its date
Activity Period	Active or Not Active between two dates, without editing the schedule itself
Retroactive Run	Whether to run a missed day after the server was down for more than a day

Cyclic jobs

A cyclic job or SMART folder runs several times within one order.

- **Interval:** every N minutes/hours/days, measured from the Start, End or Target of the previous run.
- **Interval Sequence:** a list of varying gaps, e.g. +10, +20, +30 minutes.
- **At Specific Times:** e.g. 08:00, 12:00, 16:00, with a **Tolerance** in minutes for late starts.
- **Maximum Reruns:** 0-99 on distributed systems. Use the **Stop Cyclic Run** if-action to end the cycle on a condition.

Watch out

Cyclic jobs stay in Monitoring until the cycle ends or the job is removed. Always pair them with a sensible To time or Keep Active value, or they can run far longer than intended.

4. Calendars

Type	Use it for
Regular calendar	A list of working (or non-working) dates, e.g. a national holiday calendar
Periodic calendar	Business periods labelled A, B, C..., e.g. accounting periods that don't match months
Rule-Based calendar (RBC)	Rules such as 'Mon-Fri' or 'last working day', reusable across folders
Folder RBC	An RBC defined inside one SMART folder, used only by its sub-folders and jobs

Confirmation calendars

An RBC can be filtered by a **confirmation calendar** (a regular or periodic calendar). When a run date in the RBC is not a working day in the confirmation calendar, the **Exception Policy** decides what happens: disable the run, move to the next or previous confirmed day, or ignore the confirmation calendar. **Shift By** (-62 to +62 days) moves runs further.

Example: RBC 'Production Schedule' = Mon-Fri; confirmation calendar 'Holidays'; policy 'Run on Next Confirmed Day'. A job due on a public holiday runs on the next working day instead.

5. Prerequisites: controlling flow

Events (dependencies)

A predecessor job **adds** an event when it ends; the successor **waits** for that event. Events carry a date (ODAT by default), so today's run only satisfies today's successor. Drawing a link between jobs in a workspace creates these events for you. Successors usually **delete** the event after they start so a rerun of the predecessor doesn't trigger them twice.

- Use AND/OR logic and parentheses when a job waits on several events.
- Within SMART folders, prefer events confined to the folder so parallel instances don't trigger each other (see Run as Independent Flow).
- Cross-folder and cross-server dependencies are possible but harder to trace; document them in the job description.

Resources

Type	How it works	Example
Lock resource (Exclusive)	Only one job holds it at a time	Two jobs that both rebuild the same table
Lock resource (Shared)	Many shared holders, but blocks an exclusive holder	Readers vs. a nightly reorg
Resource pool (quantitative)	Jobs consume N units out of a total	At most 5 DB-heavy jobs in parallel

Other prerequisites

- **Confirmation:** the job waits in Wait User until an operator confirms it.
- **Priority** and **Critical:** affect which waiting job gets resources first.
- **Host / host group** (Run Where): a host group spreads load and survives one Agent being down.

6. Actions: what happens after a job runs

If-actions

If (condition)	Common actions
Job Ended Not OK	Notify, Rerun Job, Add/Delete Event (e.g. trigger a recovery job)
Job Ended OK / Job Ended	Handle Output, Add Event, Set Variable
OS Completion Status (exit code)	Set to OK for a known harmless code, e.g. robocopy exit codes 1-7
Specific Statement Output	Act on text in the output, e.g. ON "*" "*not found*"
Job's Number of Reruns / Failures	Stop retrying after N attempts and notify
Variable Value	Branch the flow on a captured value
Output Cannot Be Found	Notify the Alerts window

Available actions: Notify, Set to OK, Set to Not OK, Rerun Job, Stop Cyclic Run, Set Variable, Run Job and Ignore Scheduling Criteria, Handle Output (copy/move/delete/print), Add/Delete Event.

Windows example

Robocopy returns 1-7 for success-with-changes. Without an If-action 'OS Completion Status 1-7 > Set to OK', those runs end Not OK and page someone at night for nothing.

Notifications

- **Before completion:** Job Not Submitted by <time>, Job's Execution Time exceeds N minutes (or deviates from the average), Job Not Finished by <time>, Job Set to Rerun, cyclic submission late by N minutes.
- **After completion:** Job Ended OK / Job Ended Not OK.
- Destinations: Alerts window, Mail, Remedy ticket, Job Log, User Console, or a named destination list. Urgency: Normal, Urgent, Critical.

Capturing output into variables

Capture from Job Output finds a string in the job's output and saves the next word(s) or the rest of the line into a variable. A later job, or an If-action on Variable Value, can then use it, for example a file name or record count.

7. Variables

Kind	Scope / example
System variables	Built-in values such as %%ODATE (order date), %%JOBNAME, %%RUNCOUNT, %%ORDERID
Local (job) variables	Defined in one job; passed to the script as parameters or environment
SMART folder variables	Shared by all jobs in one folder instance, e.g. a batch date for the whole flow
Global / named pool variables	Shared across folders, e.g. a feature switch or a last-processed key

```
REM Job command line using variables
powershell.exe -File D:\scripts\load_sales.ps1 -BusinessDate %%ODATE -Run %%RUNCOUNT
```

8. Jobs-as-code with the Automation API

Definitions can be kept as JSON in Git and deployed with the ctm CLI or REST API. This gives you review, history and repeatable promotion from dev to prod.

```
{
  "FIN_DAILY": {
    "Type": "Folder",
    "ControlmServer": "IN01",
    "Application": "FIN", "SubApplication": "GL",
    "When": { "WeekDays": ["MON","TUE","WED","THU","FRI"], "FromTime": "0200" },
    "Extract": {
      "Type": "Job:Command",
      "Command": "powershell -File D:\\fin\\extract.ps1 %%ODATE",
      "Host": "hg-fin-win", "RunAs": "svc_ctm_fin"
    },
    "Load": {
      "Type": "Job:Command",
      "Command": "powershell -File D:\\fin\\load.ps1",
      "Host": "hg-fin-win", "RunAs": "svc_ctm_fin"
    },
    "Flow": { "Type": "Flow", "Sequence": ["Extract", "Load"] }
  }
}
```

```
ctm build FIN_DAILY.json      # validate only
ctm run    FIN_DAILY.json     # test-run without deploying
ctm deploy FIN_DAILY.json     # publish definitions
ctm run order IN01 FIN_DAILY  # order the folder now
```

Use a **Deploy Descriptor** to swap server, host group and run-as user as JSON moves dev to prod.

9. Design checklist

Area	Good practice
Naming	Application prefix on folders and jobs; enforce with Site Standards
Structure	One SMART folder per business flow; sub-folders for stages
Hosts	Target host groups, not single hosts, for anything critical
Run as	Dedicated service accounts per application, never personal accounts
Failure handling	Exit-code rules, limited automatic reruns, then notify the right team
Late detection	'Not Submitted by' / 'Not Finished by' on critical jobs; SLA services for deadlines
Testing and docs	Validate with ctm build or a test folder; add a runbook link in Documentation

Common scheduling mistakes

- Event dates that don't match (adds ODAT, waits PREV), so the successor waits forever.
- A To time earlier than the From time without allowing submission past New Day.
- Forgetting to delete events, so the successor runs twice after a predecessor rerun.
- Cyclic jobs with no stop condition or Maximum Reruns.
- Holiday calendars not updated for next year before the New Day of 1 January.

10. Job types

The job type decides what the Agent actually does. OS jobs are the most common; plug-in job types appear once the matching plug-in is deployed on the Agent.

Job type	Automation API type	Use for
Command	Job:Command	A single command line, e.g. powershell.exe -File ...
Script	Job:Script	A script file on the host, with arguments
Embedded Script	Job:EmbeddedScript	Script text stored inside the job definition
Dummy	Job:Dummy	Flow control: start/end markers, joining branches, placeholders
File Watcher	Job:FileWatcher:Create / :Delete	Wait for a file to arrive (or disappear) before successors run
File Transfer (MFT)	Job:FileTransfer	Move files between hosts, SFTP, cloud storage
Database	Job:Database:SQLScript, :StoredProcedure, ...	Run SQL or procedures through a connection profile
SLA Management	Job:SLAManagement	Define a service with a completion deadline
Application plug-ins	e.g. Job:SAP:R3, Job:AWS..., Job:Azure...	Run work inside SAP, cloud services, data platforms
Application Integrator	Job:ApplicationIntegrator:<name>	Custom job types your admins built, e.g. around a REST API

Fields every job should have

Field	Guidance
Job name	Application prefix + purpose, e.g. FIN_GL_EXTRACT. Unique within the folder
Description	One line on what it does and the business impact if it fails
Application / Sub-application	Used for filtering viewpoints, roles, and reports. Fill them consistently
Host / host group	Prefer a host group for anything critical
Run as	The service account for the application
Documentation	URL to the runbook, so operators know what to do at 3 a.m.
Priority / Critical	Only for genuinely critical paths; overuse makes them meaningless

11. The New Day and the order date

Control-M works on a **logical business day**, not the calendar day. Each day starts at the **New Day time** configured for your Control-M/Server. At New Day, Control-M decides which jobs and folders are scheduled for the new date and orders them into the active environment.

- **ODATE** (order date) is the logical date a job was ordered for. A job ordered for 9 October still has ODATE 20261009 if it runs at 01:30 on 10 October, as long as that is before New Day.
- Scripts should take the business date from %ODATE, not from the system clock. Reruns then process the right day.
- Jobs ordered with Keep Active can stay in the active environment over several New Days, still carrying their original ODATE.

- Events carry dates too. ODAT means 'today's order date'; PREV means the previous scheduled date; STAT means a date-independent event.

Example

If New Day is 06:00, a job with From 02:00 runs at 02:00 on the *next* calendar day but within the same logical day. Confirm the New Day time with your administrator before designing overnight flows.

12. File watchers

File watcher jobs let a flow start when an input file arrives instead of at a fixed time.

Setting	Meaning
Path / file name	Folder and pattern to watch, e.g. \\fileserver\in\sales_*.csv
Search interval	How often to check for the file
Time limit	How long to wait before failing (so a missing file raises an alert instead of waiting forever)
Minimum size / size stability	Wait until the file stops growing, so you don't pick up a half-written file
Wildcard handling	Whether the found file name is passed on in a variable for the successor

- Combine with a 'Job Not Submitted by' or a time limit so a late file raises an alert at a sensible time.
- Ask the sending system to write to a temporary name and rename when complete. This is more reliable than size checks.
- Use Capture from Job Output or the file watcher's variable to pass the exact file name downstream.

13. Worked example 1: nightly finance load

Requirement: every working day (Mon-Fri, excluding Indian public holidays), extract general-ledger data at 02:00, load it into the warehouse, then refresh reports. Reports must be ready by 06:30. Only one warehouse load may run at a time across all applications.

Design element	Choice
Folder	SMART folder FIN_DAILY, Application FIN, Sub-application GL
Schedule	RBC 'WORKDAYS' (Mon-Fri) with confirmation calendar 'IN_HOLIDAYS', policy Disable Run
Jobs	FIN_GL_EXTRACT → FIN_GL_LOAD → FIN_RPT_REFRESH (events created by drawing links)
Time	FIN_GL_EXTRACT From 02:00; others No Start Time (they follow events)
Concurrency	FIN_GL_LOAD takes lock resource DWH_LOAD (Exclusive)
Host	Host group HG_FIN_WIN (two Windows Agents)
Failure handling	If Job Ended Not OK → Rerun Job, max 2 reruns, then Notify FIN-SUPPORT (Urgent)
Late detection	FIN_RPT_REFRESH: Job Not Finished by 06:15 → Notify; SLA service with Must Complete 06:30
Variables	Business date passed as %%ODATE to all three scripts

```
{
  "FIN_DAILY": {
    "Type": "Folder", "ControlmServer": "IN01",
    "Application": "FIN", "SubApplication": "GL",
    "RunAs": "svc_ctm_fin", "Host": "HG_FIN_WIN",
    "When": { "RuleBasedCalendars": { "Included": ["WORKDAYS"] } },
    "FIN_GL_EXTRACT": {
      "Type": "Job:Command",
      "Command": "powershell -NoProfile -File D:\\fin\\extract.ps1 %0DATE",
      "When": { "FromTime": "0200" },
      "IfFailed": { "Type": "If:CompletionStatus", "CompletionStatus": "NOTOK",
        "Mail": { "Type": "Action:Mail", "To": "fin-support@example.com",
          "Subject": "FIN_GL_EXTRACT failed %0DATE", "AttachOutput": true } }
    },
    "FIN_GL_LOAD": {
      "Type": "Job:Command",
      "Command": "powershell -NoProfile -File D:\\fin\\load.ps1 %0DATE",
      "Lock": { "Type": "Resource:Lock", "DWH_LOAD": { "LockType": "Exclusive" } }
    },
    "FIN_RPT_REFRESH": {
      "Type": "Job:Command",
      "Command": "powershell -NoProfile -File D:\\fin\\refresh.ps1 %0DATE"
    },
    "Flow": { "Type": "Flow",
      "Sequence": ["FIN_GL_EXTRACT", "FIN_GL_LOAD", "FIN_RPT_REFRESH"] }
  }
}
```

Validate first

The JSON keys above follow Automation API conventions. Validate with `ctm build` before deploying, because property names and nesting must match your API version exactly.

14. Worked example 2: file-triggered partner feed

Requirement: a bank drops `stmt_YYYYMMDD.csv` on SFTP between 18:00 and 22:00. When it arrives, fetch it, validate it, and load it. If it hasn't arrived by 22:00, alert the payments team.

1. **PAY_STMT_WATCH** (File Transfer or File Watcher job against the bank's SFTP connection profile): From 18:00, time limit until 22:00.
2. Notification on **PAY_STMT_WATCH**: Job Ended Not OK → Notify PAY-SUPPORT 'Bank statement not received by 22:00' (Critical).
3. **PAY_STMT_GET** (File Transfer job): download to `\\payfs\in\`, using PGP decryption if the bank encrypts the file.
4. **PAY_STMT_VALIDATE**: script checks row count and trailer; exit 0 OK, exit 2 = data error.
5. If-action on **PAY_STMT_VALIDATE**: OS Completion Status 2 → Notify PAY-SUPPORT and Set to Not OK (so it is clearly a data issue, not an infrastructure one).
6. **PAY_STMT_LOAD**: loads to the payments database with a Database job type and connection profile.

15. Worked example 3: intraday cyclic polling

Requirement: between 08:00 and 20:00 on working days, process new orders every 15 minutes. Stop for the day if three runs in a row fail.

Setting	Value
Cyclic	On; Interval 15 minutes from the End of the previous run
From / To	08:00 / 20:00
Maximum Reruns	48 (12 hours x 4 per hour)
If-action	Job's Number of Failures = 3 → Stop Cyclic Run + Notify ORD-SUPPORT (Urgent)
Notification	Job's Calculated Cyclic Submission Is Late By 10 minutes → Alerts window
Keep Active	0, so it does not carry over into the next day

Measuring the interval from the **End** of the previous run means a slow run never overlaps the next one. Measuring from **Start** keeps a fixed rhythm but can queue runs if one takes longer than 15 minutes.

16. Services and SLA management

A **service** represents a business outcome, such as 'Finance reports ready', rather than an individual job. Using statistics from previous runs, Control-M estimates whether the whole chain will finish on time and warns before the deadline is missed, not after.

- Add an **SLA Management job** at the end of the critical flow and set **Must Complete At** (e.g. 06:30) and the service's priority.
- Control-M follows the jobs that lead into it and shows the service as On Time, At Risk, Late or Failed in Monitoring.
- Use **SLA Manager / Forecast Rules** to adjust expected run times for known exceptions, such as month-end jobs that always take longer.
- Use **Periodic Statistics** rules so month-end runs are compared with past month-ends rather than ordinary days.

17. Workload Policies and templates

Feature	Use it to
Workload Policies	Limit concurrent jobs by application, host or job type; pause a group of jobs during maintenance without editing each job
Job and folder templates	Pre-fill standard values (host group, run-as, notifications, application) so new jobs follow standards
Folder Management	See all folders across environments and find what exists before building something new
Site Standards	Block check-in of definitions that break naming or property rules

18. Variables: functions and patterns

Expression	Result / use
%%ODATE	Order date, YYMMDD or YYYYMMDD depending on the variable used (%%\$ODATE for 4-digit year)
%%\$CALCDATE %%\$ODATE -1	The day before the order date, e.g. to process yesterday's files
%%SUBSTR %%\$ODATE 1 6	Substring, here year + month (YYYYMM)
%%JOBNAME, %%ORDERID, %%RUNCOUNT	Identify the run inside log lines and file names
%%A %%B	Concatenation with the %%. operator
%%\MYVAR	Global variable reference

Typical patterns: build file names from dates (sales_%%\$ODATE.csv), pass a 'mode' variable to the same script for full vs. incremental runs, or capture a record count from one job's output and branch on it in the next.

19. Time zones

- By default a job follows the time zone of the Control-M/Server's logical day.
- Set the job's **Time Zone** when it must run at a local time elsewhere, e.g. 09:00 America/New_York for a US partner.
- Watch daylight-saving changes: a job scheduled in a zone that changes clocks will shift relative to India time twice a year.
- Keep related jobs in one flow in the same time zone where possible. Mixed zones make dependencies hard to reason about.

20. Pre-check-in review checklist

Check	Why
Calendar and RBC give the expected dates (use View Schedule)	Catches wrong weekdays and missing holidays
From / To / Keep Active make sense together	Prevents jobs never starting or lingering for days
Every event added is also waited for, and the dates match	Prevents orphan events and stuck successors
Successors delete their in-conditions where reruns are possible	Prevents double runs
Failure path defined: exit-code rules, rerun limit, notification	Operators know what happened and who owns it
Host group, run-as, application fields filled	Routing, security and reporting work
Runbook link in Documentation	Operators can act without calling you
Validated (ctm build / test folder)	Catches syntax and reference errors before production

21. Glossary

Term	Meaning
Active environment	Jobs ordered for execution (what you see in Monitoring)
Check in / check out	Publish workspace changes / take definitions into a workspace for editing
Confirmation calendar	Calendar that filters an RBC's dates, with an exception policy
Cyclic	Job or folder that runs repeatedly within one order
Event (condition)	Dated flag added by one job and awaited by another
Keep Active	Days a job stays waiting after its order date
New Day	Daily point where Control-M orders the next logical day's jobs
ODATE	Logical order date of a job
RBC	Rule-Based Calendar
Resource pool	Quantity-based resource that limits how many jobs run at once
SMART folder	Folder treated as one unit with its own scheduling, status and actions

Based on BMC's Control-M SaaS documentation (documents.bmc.com/supportu/controlm-saas), checked October 2026. Control-M SaaS was previously called BMC Helix Control-M. Menu names and limits change between releases, so confirm details against your tenant and the current docs.